1. Code in Mosaic

This example collects the supported ways to put code-like text in a Mosaic document. Inline code uses backticks, as in `cargo mos check` or `#set text(font: "Noto Sans")`.

Inline code can also span a soft source line break when a short phrase needs to stay visually code-shaped in the editor and output: `a sad student wrote this`.

The source forms for those inline examples are:

```
`cargo mos check`
`#set text(font: "Noto Sans")`

`a sad student wrote
this`
```

1.1. Preformatted command transcripts

Use `#pre[[...]]` for terminal output, command transcripts, or other preformatted text where the content is not tied to a programming language.

```
$ cargo mos check examples/code/main.mos
$ cargo mos build examples/code/main.mos
Finished `dev` profile
```

The Mosaic source for that transcript block is:

```
#pre[[
$ cargo mos check examples/code/main.mos
$ cargo mos build examples/code/main.mos
Finished `dev` profile
]]
```

Arguments are accepted before the raw body. The layout currently keeps the raw text literal; semantic handling of arguments is future work.

```
step one
step two
```

The Mosaic source for that argument form is:

```
#pre(indent: 2)[[
step one
step two
]]
```

Raw blocks can be empty: `#pre[[]]`.

1.2. Code blocks

Use `#code[[...]]` for code examples. Mosaic preserves the raw block text and renders it with the monospace face.

```
fn main() {
```

```
    println!("hello, mosaic");
}
```

The Mosaic source for that language-tagged code block is:

```
#code(lang: "rust")[[
fn main() {
    println!("hello, mosaic");
}
]]
```

Code blocks can omit arguments when no language metadata is needed.

```
let answer = 42;
```

The Mosaic source for the bare code form is:

```
#code[[
let answer = 42;
]]
```

Raw blocks can carry trailing labels.

```
let labelled = true;
```

The Mosaic source for a labelled raw block is:

```
#code[[
let labelled = true;
]] <ex:labelled-code>
```

Code blocks are raw long-bracket bodies. A plain `[[...]]` body can contain single closing brackets and backslash-bracket text literally.

```
let vector = vec![1, 2, 3];
let bracket = "]";
let slash_close = "\\]";
println!("{vector:?} {bracket} {slash_close}");
```

The Mosaic source for literal `]` and `\]` text is:

```
#code(lang: "rust")[[
let vector = vec![1, 2, 3];
let bracket = "]";
let slash_close = "\\]";
println!("{vector:?} {bracket} {slash_close}");
]]
```

When the body needs `]]`, add matching equals signs to the delimiter: `[=[...]=]`.

```
let literal = "single ] and pair ]] are both fine";
println!("{literal}");
```

The Mosaic source for a block containing `]]` is:

```
#code(lang: "rust")[=[
let literal = "single ] and pair ]] are both fine";
println!("{literal}");
]=]
```

If the body itself needs `]=]`, use one more equals sign: `[==[...]==]`.

```
let delimiter_text = "this source contains ]=] literally";
println!("{delimiter_text}");
```

The Mosaic source for a block containing `]=]` is:

```
#code(lang: "rust")[==[
let delimiter_text = "this source contains ]=] literally";
println!("{delimiter_text}");
]==]
```

1.3. Choosing a form

Use inline code for identifiers and small fragments such as `NodeKind::Raw`. Use multiline inline code only for short fragments that wrap in the source. Use `#pre[[...]]` for pasted terminal sessions. Use `#code[[...]]` for source snippets that should stand on their own. Increase the number of equals signs when the block body contains the current closing delimiter.